

Development Summary

1. Project Overview

We have successfully built ExpenseTrack, a full-stack web application that allows users to securely log in, record personal expenses, and generate spending reports through an intuitive web interface. The system consists of a secure backend API for data handling and authentication, and a responsive frontend UI for seamless user interaction, fulfilling all core requirements specified.

2. Implemented Services

Backend API

Handles user authentication, secure expense management, and report generation with robust data validation and user isolation.

- `app/api/auth.py` – Implements user signup, login, and logout using JWT-based authentication.
- `app/core/security.py` – Provides secure password hashing with `bcrypt` and JWT token creation.
- `app/api/expenses.py` – Manages CRUD operations for expenses with user ownership checks and category validation.
- `app/api/reports.py` – Generates monthly and category-wise spending reports scoped to the authenticated user.
- `requirements.txt` – Lists all Python dependencies including FastAPI, SQLAlchemy, `bcrypt`, and JWT support.

Frontend UI

A React-based user interface enabling users to add and manage expenses with a clean, modern design.

- `src/pages/AddExpense.jsx` – Interactive form for creating new expenses with auto-filled date and category selection.

3. Local Setup Instructions

To run ExpenseTrack locally, follow these steps:

1. Ensure you have Python 3.9+ and Node.js 16+ installed on your machine.

2. Set up a PostgreSQL database and configure the DATABASE_URL environment variable accordingly.

3. Install Python dependencies by running:

```
`bash  
pip install -r requirements.txt  
`
```

4. Navigate to the frontend directory and install Node.js dependencies:

```
`bash  
cd frontend && npm install  
`
```

5. Create a .env file in the backend root with the following variables:

```
`  
SECRET_KEY=your_strong_secret_key  
DATABASE_URL=your_postgres_database_url  
ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=30  
`
```

6. Start the backend server:

```
`bash  
uvicorn app.main:app --reload  
`
```

7. In a separate terminal, start the frontend:

```
`bash  
cd frontend && npm start  
`
```

The application will be accessible at <http://localhost:3000>.

4. Development Notes

- Passwords are securely hashed using bcrypt, and JWT tokens are used for stateless authentication.
- All expense and report operations are scoped to the authenticated user, preventing unauthorized data access.
- Expense categories are validated against a predefined list to ensure data consistency.
- The "Add Expense" form defaults the date to the current day for better user experience.

- For production deployment, JWT tokens should be served via httpOnly cookies to enhance security against XSS attacks.
- The current setup uses a simple in-memory session model for JWT; a token revocation strategy may be added later if needed.